웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202446124 이름: 김민영 제출일: 2026 년 10 월 28 일

서비스 이름: 모동숲(닌텐도 게임) 교환 정보 게시판

기술 조합과 버전 [Vue / Spring Boot / Windows 11, JDK 27, Node.js v24.21.0

GitHub 저장소 URL https://github.com/Miy329/web_service_hw2

소스 코드 기준 main 브랜치 / 최종 제출 기준 커밋 SHA 는 LMS 에 제출

사용자와 사용 목적

이 서비스는 필자가 즐겨하는 게임의 닌텐도 스위치 게임인 "모여봐요 동물의 숲"을 하는 사용자들이 필요한 아이템을 구하거나 남는 아이템을 나눔 하고, 섬 방문과 같은 게임 정보를 주고받을 수 있는 게시판이다. 평소 아이템을 교환할 때 게임 유저들만의 소통전용 서비스가 있으면 편리하겠다는 생각을 하였고 그 생각을 실천해보고자 해당 서비스를 선택했다. 서비스의 기능은 로그인한 사용자만이 글작성이 가능하고 수정과 삭제는 글을 쓴 본인만 할 수 있도록 하였다.

기술 조합은 Vue 와 Spring Boot 를 선택하였다. Spring Boot 는 이전 백엔드 실습 과제에서 사용해 본 경험이 있어 구조가 익숙했고, Vue 는 화면과 동작 코드가 한 파일에 모여 있어 요청 흐름을 따라가기 쉬웠기 때문이다.

제공 템플릿에서 서버는 AuthService.java 의 토큰 발급, 토큰 검증, 작성자 비교 부분을 구현하였고, 화면은 api.js 의 로그인, 토큰 전송, 401 처리, 수정 버튼 표시 조건과 App.vue 의 오류 안내 문구 및 화면 제목을 수정하였다. 게시글 저장, 입력 검사, 컨트롤러는 제공된 코드를 그대로 사용하였다.

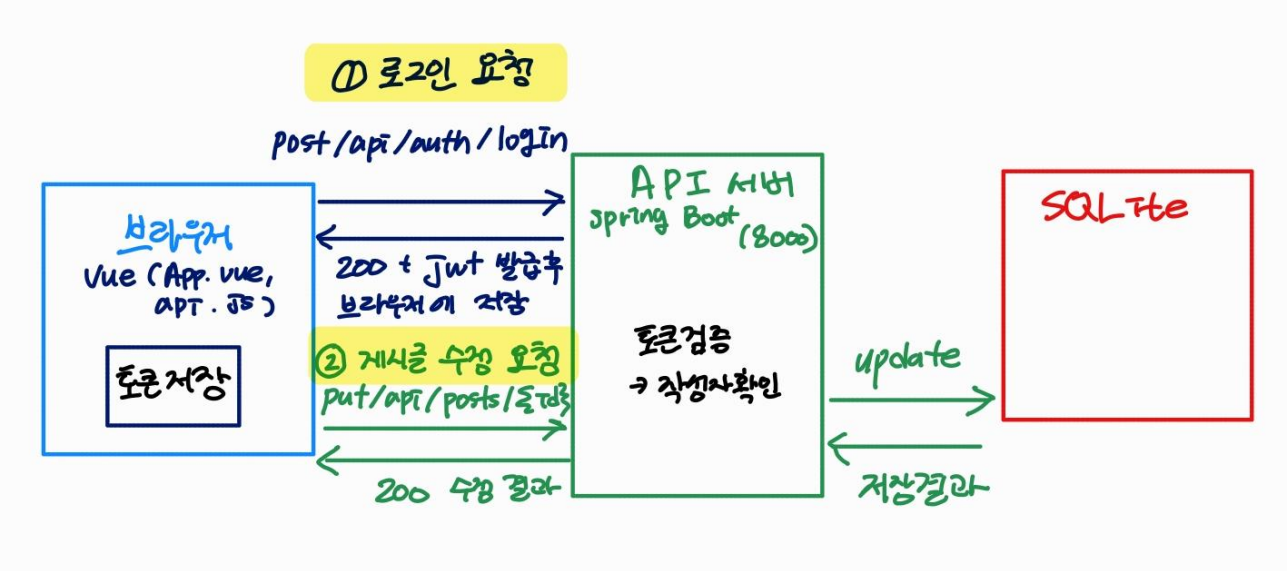
기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.java, App.vue	제공 코드 유지
로그인과 JWT 발급	완료	AuthService.java, api.js	토큰 유효 시간 기본 90 초
인증과 내 정보 조회	완료	AuthService.java, BoardController.java	서명, 만료, 발급자, 사용 대상 검증
본인 글 작성·수정·삭제	완료	AuthService.java, api.js	다른 사람 글은 서버에서 403 으로 거부
만료·로그아웃·오류 표시	완료	api.js, App.vue	로그아웃은 현재 탭의 토큰만 삭제

기능	완료 여부	관련 파일	비고 또는 제한
DB 데이터 유지	완료	PostStore.java, data/board.sqlite	제공 코드 유지

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할

아래의 그림은 브라우저 → API 서버 → SQLite 의 연결 구조의 그림이다. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하였다.



이 서비스는 브라우저(4173 포트), API 서버(8000 번 포트), SQLite 파일의 DB 로 구성된다. 브라우저에서 보낸 api 요청은 서버로 전달되고, 서버는 SQLite 에서 회원과 게시글 정보를 조회하거나 저장한다.

로그인 요청은 토큰 없이 이메일과 비밀번호를 POST /api/auth/login 으로 보낸다. 서버는 계정을 확인한 뒤 JWT 를 발급해 응답하고, 브라우저는 이 토큰을 api.js 의 session 변수, 즉 메모리에 저장한다.

게시글 수정 요청은 PUT /api/posts/{id}에 Authorization: Bearer 헤더로 저장해 둔 토큰을 함께 보낸다. 서버는 토큰을 검증해 요청한 사용자를 확인하고, 글이 있는지와 작성자가 맞는지 확인한 뒤에만 SQLite 의 게시글을 수정한다. 토큰은 내 정보 조회, 글 등록, 수정, 삭제 요청에서 사용되며, 목록과 상세 조회는 토큰 없이도 동작한다.

변경한 파일과 함수 경로/함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유	출처
AuthService.java /checkSecret(), issueToken(), key()	토큰 발급 부분이 비어 있음	로그인한 사용자 id, 발급자, 사용 대상, 발급 시간과 만료 시간을 담아 서명한 토큰을 발급하도록 구현하였다. 서버가 켜질 때 비밀 키 형식도 검사하도록 하였다.	로그인한 사용자에게만 위조할 수 없는 토큰을 주고, 잘못된 키로 서버가 실행되지 않게 하기 위함이다.	AI 초안 후 본인 수정
AuthService.java / assertOwner()	비교 없이 통과	요청한 사용자와 글 작성자가 다르면 403 FORBIDDEN 을 반환하도록 구현하였다.	화면과 관계없이 본인 글만 수정, 삭제되게 하기 위함이다.	AI 초안 후 본인 수정
api.js / login(), authorizationHeaders()	로그인 연결이 구현되지 않았고 토큰 헤더가 없음	로그인 응답의 토큰과 사용자 정보를 메모리에 저장하고, 토큰이 있을 때만 Bearer 헤더를 붙이도록 하였다.	토큰을 메모리에만 보관하고 보호 API 요청에 함께 보내기 위함이다.	AI 초안 후 본인 수정
api.js / request(), canEdit()	401 처리가 없고 모든 글에 수정 버튼을 표시	401 을 받으면 로그인 정보를 지우고, 본인 글에만 수정, 삭제 버튼을 표시하도록 하였다.	토큰이 만료되거나 잘못되었을 때 화면도 로그아웃되게 하고, 다른 사람 글에는 버튼이 보이지 않게 하기 위함이다.	AI 초안 후 본인 수정
App.vue / run(), errorMessage(), save(), logout()	서버의 오류 문구를 그대로 표시	code 별 안내 문구를 추가하고, 404 면 목록을 새로 불러오며, 로그인 전 글쓰기 시 안내를 띄우고, 로그아웃 시 입력창을 비우도록 하였다.	사용자가 무엇이 잘못되었는지 알 수 있게 하기 위함이다.	본인 작성
App.vue / 화면 제목과 안내 문구	예시 제목 "우리의 기록"이라는 문구가 존재하였다.	서비스 이름과 서비스 안내 문구로 변경하였다.	서비스 주제에 맞게 하기 위함이다.	본인 작성
.gitignore	빌드 결과물과 DB 파일 등을 제외	.idea/를 추가하였다.	개발 도구의 설정 파일이 저장소에 올라가지 않게 하기 위함이다.	본인 작성
AuthService.java / currentUserId()	토큰을 검사하지 않고 임시 사용자 id 를 반환	헤더에서 토큰을 꺼내 서명, 만료, 발급자, 사용 대상, 사용자 존재 여부를 확인하고, 실패 이유에 따라 401 code 를 나누도록 구현하였다.	요청한 사용자를 서버가 직접 확인하고, 위조되거나 만료된 토큰을 막기 위함이다.	AI 초안 후 본인 수정

그 밖의 파일은 제공된 코드 그대로 유지하였다.;

로컬 실행 기록

운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로

자세한 재현 방법은 저장소 README.md 참고

항목	내용
운영체제와 버전	Windows 11 , JDK 27, Node.js v24.21.0
새 폴더 설치	imsi_web₩JWT-Assignment 에 과제 파일을 받아 설치
환경설정	cmd 에서 set JWT_SECRET=<새로 만든 64 자리 16 진수 키> 실행 (실제 키는 기록하지 않았음), TOKEN_TTL 기본값 90 초
프론트 빌드	npm ci, npm run build 실행, 빌드 성공
백엔드 실행	mvnw.cmd package 실행 후 BUILD SUCCESS, java -jar target₩jwt-board-1.0.0.jar 실행, 127.0.0.1:8000 에서 동작
SQLite 파일 경로	backend-spring/data/board.sqlite (서버 실행 시 자동 생성)
4173 번 접속 및 API 연결 확인	http://127.0.0.1:4173 접속 후 Network 에서 GET /api/posts 200 확인
서버 재시작 후 데이터 유지	서버를 종료한 뒤 다시 실행해도 게시글과 작성자가 그대로 유지됨을 확인
증거 위치	evidence/Build_success.png, evidence/T01.png, evidence/T15_before_serverOff.png, evidence/T15_2_after_on.png

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java / issueToken(), checkSecret(), key()

코드 발췌

```
// BoardController.java 의 login()
var input = fields(body, Map.of("email", 254, "password", 128));
var user = users.verify(input.get("email"), input.get("password"));
if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);

// AuthService.java 의 issueToken() 함수
public String issueToken(Users.PublicUser user) {
  Date now = new Date();
  Date expiration = new Date(now.getTime() + ttl * 1000);
  return Jwts.builder()
          .subject(String.valueOf(user.id()))
          .issuedAt(now)
          .expiration(expiration)
          .issuer(issuer)
          .audience().add(audience).and()
          .signWith(key(), Jwts.SIG.HS256)
          .compact();
```

동작과 응답 코드 설명

입력 및 검증: 제공 코드의 fields()가 화면에서 보낸 JSON 본문의 email 과 password 가 문자열인지, 길이가 제한(254 자, 128 자)을 넘지 않는지, 다른 필드가 섞여 있지 않은지 검사한다. 형식이 맞지 않으면 400 BAD_INPUT 을 반환한다.

계정 확인: users.verify()가 저장된 비밀번호 해시와 입력한 비밀번호를 비교한다. 비밀번호가 틀리거나 없는 이메일이면 null 을 돌려주고, login()이 401 INVALID_CREDENTIALS 를 보낸다. 보안을 위해 두 경우 모두 같은 code 를 사용하여 어느 쪽이 틀렸는지 알려 주지 않도록 하였다.

토큰 발급: 계정 확인을 통과한 사용자에게만 issueToken()이 호출된다. 토큰에 사용자 id(sub), 발급자(iss), 사용 대상(aud), 발급 시간(iat), 만료 시간(exp)을 담았고 비밀번호는 넣지 않았다. 만료 시간은 현재 시간에 TOKEN_TTL(기본 90 초)을 더해 정하였으며, 서버 비밀 키로 HS256 서명을 해서 토큰을 완성한다.

비밀 키: 서명용 키는 코드에 직접 쓰지 않고 환경변수 JWT_SECRET 에서 읽도록 하였다. checkSecret()은 서버가 켜질 때 한 번 실행되어, 키가 64 자리 16 진수가 아니면 예외를 던져 서버가 실행되지 않게 막는다.

출력: 성공하면 200 과 함께 accessToken, expiresIn(90), user(id, name, email)를 돌려준다.

3.2 브라우저 토큰 저장과 전송

파일 / 함수:

1: frontend-vue/src/api.js / login(), authorizationHeaders(), request()

2: frontend-vue/src/App.vue / logout()

코드 발췌

```
// api.js
let session = { accessToken: "", user: null };
function authorizationHeaders() {
  if (!session.accessToken) return {};
  return { Authorization: `Bearer ${session.accessToken}` };
}

// request() 안: 실패 응답 처리
if (!response.ok) {
  if (response.status === 401) clearSession();
}

export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}

// App.vue / logout()
clearSession();
user.value = null;
```

동작과 응답 코드 설명

로그인 응답 처리 및 저장 위치: login()은 이메일과 비밀번호를 POST /api/auth/login 으로 보낸다. 200 응답을 받으면 토큰과 사용자 정보를 session 변수(메모리)에 저장하고, 화면(App.vue)은 이 정보로 로그인 상태를 표시한다. 과제 조건에 맞춰 localStorage 에 저장하지 않았으므로 새로고침 시 로그인이 해제되지만, 게시글은 서버 DB 에 있어 유지된다.

Authorization 헤더: 모든 요청은 request()를 거치며, authorizationHeaders()가 Authorization: Bearer <JWT> 헤더를 붙인다. 토큰이 없을 때는 빈 객체를 반환하여 잘못된 헤더(Bearer undefined)가 전송되지 않게 하였다.

401, 403 처리: 서버가 401 을 보내면 request() 안에서 clearSession()을 호출해 메모리의 토큰과 사용자 정보를 지우고, 화면은 로그아웃 상태로 바뀌며 code 에 맞는 재로그인 안내를 보여 준다. 403 은 로그인은 유효하고 권한만 없는 경우라서 로그아웃시키지 않는다.

로그아웃: 로그아웃 버튼은 clearSession()으로 이 탭의 토큰과 사용자 정보를 지우고 화면 표시를 즉시 바꾼다. 다만 이미 복사된 토큰을 서버에서 무효화하는 기능은 아니어서, 그 토큰은 만료 시각까지는 서버에서 유효하다.

웹서비스설계 03 계속

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java / currentUserId()

코드 발췌

```
String header = request.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
  throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
}
String token = header.substring(7).trim();

Claims claims;
try {
  Jws<Claims> jws = Jwts.parser()
          .verifyWith(key())
          .requireIssuer(issuer)
          .requireAudience(audience)
          .build()
          .parseSignedClaims(token);
  if (!"HS256".equals(jws.getHeader().getAlgorithm())) {
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  }
  claims = jws.getPayload();
} catch (ExpiredJwtException e) {
  throw new ApiError(401, "TOKEN_EXPIRED", "로그인이 만료되었습니다. 다시 로그인하세요.");
} catch (JwtException | IllegalArgumentException e) {
  throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
}

String sub = claims.getSubject();
if (claims.getExpiration() == null || sub == null || !sub.matches("[1-9][0-9]{0,15}")) {
  throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
}
long id = Long.parseLong(sub);
if (users.find(id) == null) {
```

```
  throw new ApiError(401, "INVALID_TOKEN", "사용자를 확인할 수 없습니다.");
}
return id;
```

동작과 응답 코드 설명

헤더 확인: 보호 API 는 처리 전 항상 currentUserId()를 호출한다. Authorization 헤더가 없거나 Bearer 형식이 아니면 401 TOKEN_REQUIRED 를 반환한다.

서명, 알고리즘, 필수 클레임 검사: verifyWith(key())로 서버의 비밀 키를 사용해 서명을 검증한다. 서명을 직접 확인하므로 위조된 토큰은 JwtException 을 발생시키며 401 INVALID_TOKEN 을 반환한다. 알고리즘이 HS256 이 아니거나 발급자, 사용 대상이 다를 때도 동일하게 401 INVALID_TOKEN 으로 처리된다.

만료 검사: 시간이 지난 토큰은 ExpiredJwtException 예외를 던지며 401 TOKEN_EXPIRED 를 반환한다. (401 응답에는 공통적으로 WWW-Authenticate: Bearer 헤더가 포함된다.)

사용자 ID 검사: 토큰의 sub 클레임이 올바른 숫자 형식인지, 그리고 DB(users.find(id))에 그 사용자가 실제로 있는 지까지 최종 확인한다. 존재하지 않으면 401 INVALID_TOKEN 을 반환한다. 모든 검증을 통과하면 사용자 id 를 반환하고 컨트롤러는 이 id 를 식별자로 사용한다.

3 4 작성자 권한 검사

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java / assertOwner()

코드 발췌

```
// AuthService.java / assertOwner()
public void assertOwner(long userId, long authorId) {
  if (userId != authorId) {
    throw new ApiError(403, "FORBIDDEN", "본인 글만 수정·삭제할 수 있습니다.");
  }
}

// BoardController.java / update()
long user = auth.currentUserId(req), pid = postId(id);
var p = existing(pid);
auth.assertOwner(user, p.authorId());

// BoardController.java / create()
long user = auth.currentUserId(req);
var input = fields(body, Map.of("title", 100, "content", 5000));
```

```
var p = store.create(input.get("title"), input.get("content"), user);
```

동작과 응답 코드 설명

권한 비교: assertOwner()는 currentUserId()로 확인된 "요청 사용자 ID"와 DB 에 저장된 "게시글 작성자 ID"를 비교한다. 두 값이 다르면 403 FORBIDDEN 예외를 던진다. 이 검증은 DB 수정/삭제 로직 이전에 실행되어, 한이 없는 요청은 글을 바꾸지 못한다.

401 과 403 의 차이: 401 은 토큰이 잘못되어 요청자가 누구인지 알 수 없는 경우이며, 403 은 누구인지는 확인했으나 '해당 자원에 대한 권한이 없는 경우'이다. 예를 들어 타인의 글을 정상 토큰으로 수정 시도하면 403, 토큰 없이 시도하면 401 이 반환된다.

작성자 위조 방지: 글 작성(create()) 시 클라이언트가 보낸 JSON 본문의 값을 믿지 않고, 서버가 토큰에서 직접 검증한 currentUserId()의 결과(id)를 작성자로 DB 에 저장한다. 또한 본문에 작성자 ID 등을 임의로 추가하여 전송해도 fields() 메서드가 허용된 필드(title, content) 이외의 값은 400 BAD_INPUT 으로 거부하기 때문에 타인 명의로 글을 다른 사람 이름으로 글을 쓸 수 없다.

응답 결과: 본인 글 수정 성공 시 200 OK 와 수정된 데이터를, 본인 글 삭제 시 본문 없이 204 No Content 를 반환한다. 없는 글 요청 시 404 NOT_FOUND 가 반환된다.

테스트 결과

실행 환경: Windows 11 / JDK 27, Node.js v24.21.0/ Chrome / 백엔드: java -jar, 프론트: npm run preview

로컬 배포 확인: 새 폴더 C:₩imsi_web₩JWT-Assignment 에 과제 파일을 받아 설치 / frontend-vue 에서 npm install, npm run build 성공 (vite v8.3.4, built in 72ms, evidence/Build_success.png) / backend-spring 에서 .₩mvnw.cmd package 후 java -jar target₩jwt-board-1.0.0.jar 실행 / http://127.0.0.1:4173 접속 후 Network 에서 GET /api/posts 200 확인 (evidence/T01.png)

테스트 일시 및 코드 버전: 2026 년 10 월 28 일 / 커밋 6c6bd0a

토큰 TTL: 일반 테스트 600 초 / 만료 테스트 3 초

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인, 브라우저에서 목록과 글 6 상세 조회	200, 공개 조회	200 / 해당 없음	PASS	evidence/T01.png, evidence/T01_2.png
T02	Alice 로 로그인	200, JWT 와 사용자 정보가 화면에 표시됨	200 / 해당 없음, "Alice 님 로그인 중"	PASS	evidence/T02_login200ok.png, evidence/T02_response.png
T03	틀린 비밀번호, 없는 이메일(who@class.test)로 각각 로그인 시도	401 INVALID_CREDENTIALS	401 /INVALID_CREDENTIALS	PASS	evidence/T03_wrongpw.png, evidence/T03_wrongID.png
T04	토큰 없이 curl 로 GET/api/me, POST/api/posts	401 TOKEN_REQUIRED	401/ TOKEN_REQUIRED	PASS	evidence/T04_cmd_notoken.png
T05	Alice 토큰으로 글 작성 후 /me 조회	201, authorId 1, /me 는 Alice	201 / 해당 없음, Location: /api/posts/7, authorId 1, authorName Alice. /me 는 200 / 해당 없음, Alice	PASS	evidence/T05.png, evidence/T05_2.png
T06	Alice 가 화면에서 본인 글 8 번 수정 PUT /api/posts/8, 삭제 DELETE /api/posts/8	200 및 204, 결과가 DB 에 반영	수정 200 / 해당 없음, 삭제 204 / 해당 없음 (본문 없음). 수정 내용이 목록에 반영되고, 삭제 뒤 목록에서 글이 사라짐	PASS	evidence/T06_put.png, evidence/T06_delete.png
T07	cmd 에서 Bob 의 정상 토큰으로 Alice 글 7 번 수정 PUT /api/posts/7, 다시 GET /api/posts/7	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN. 재조회 200, 제목 "무팔앙요"와 updatedAt 그대로	PASS	evidence/T07.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T08	cmd 에서 Bob 의 정상 토큰으로 Alice 글 7 번 삭제 DELETE /api/posts/7, 다시 GET /api/posts/7	403 FORBIDDEN, 글 유지	403 / FORBIDDEN. 재조회 200, 글 그대로 있음	PASS	evidence/T08.png
T09	Alice 토큰의 서명 첫 글자를 바꾼 토큰으로 GET /api/me, POST /api/posts, PUT 과 DELETE /api/posts/7, 다시 GET /api/posts/7	401 INVALID_TOKEN, 데이터 변경 없음	401 / INVALID_TOKEN (네 요청 모두), WWW-Authenticate: Bearer. 재조회 200, 글 그대로. 캡처에는 PUT, DELETE, 재조회가 보이고 /me, POST 결과는 위쪽이 잘림	PASS	evidence/T09.png
T10	TOKEN_TTL=3 으로 서버를 켜고 Alice 토큰을 받은 뒤 만료 후 GET /api/me, PUT /api/posts/7. 화면에서도 만료 후 글 등록	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	401 / TOKEN_EXPIRED (/me, PUT 모두). 화면은 "로그인 시간이 만료되었습니다. 다시 로그인하세요." 표시 후 로그아웃 상태	PASS	evidence/T10.png, evidence/T10_2_browser_screen.png
T11	Alice 로그인 후 로그아웃 버튼, 이어서 내 정보 조회 GET /api/me	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED. Request headers 에 Authorization 없음. 화면에 "로그인이 필요합니다. 먼저 로그인하세요." 표시	PASS	evidence/T11.png, evidence/T11_2.png
T12	cmd 에서 Bob 토큰으로 POST /api/posts 본문에 authorId:1 추가, 다시 userId:1 추가	400 BAD_INPUT, 위조 글 미생성	400 / BAD_INPUT (두 경우 모두)	PASS	evidence/T12.png
T13	cmd 에서 Alice 토큰으로 POST /api/posts 에 빈 제목, 공백 본문, 101 자 제목을 각각 전송	400 BAD_INPUT, DB 변경 없음	400 / BAD_INPUT (세 경우 모두)	PASS	evidence/T13.png
T14	없는 글 99999 조회 GET, Alice 토큰으로 PUT, DELETE /api/posts/99999. 잘못된 ID GET /api/posts/abc	없는 글 404, 잘못된 ID 400	404 / NOT_FOUND (조회, 수정, 삭제 모두), 400 / BAD_INPUT (abc)	PASS	evidence/T14_notfound.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T15	글이 있는 상태에서 백엔드를 Ctrl+C 로 종료하고 java -jar 로 다시 실행한 뒤 4173 새로고침	기존 글과 작성자, 내용 유지	재시작 후 GET /api/posts 200 / 해당 없음. 글 6 개와 작성자, 제목 그	PASS	evidence/T15_before_serverOff.png, evidence/T15_2_after_on.png
T16	Alice 로그인 상태에서 페이지 새로고침(F5)	공개 글 유지, 메모리 로그인 상태 해제	GET /api/posts 200 / 해당 없음. 글 6 개 유지, 화면은 "로그아웃 상태"	PASS	evidence/T16.png
T17	cmd 에서 토큰 없이 PUT, DELETE /api/posts/7. Bob 계정으로 화면에서 글 등록, 본인 글 10 번 수정, 삭제	무토큰은 401, Bob 본인 작업은 성공	무토큰 401 / TOKEN_REQUIRED (PUT, DELETE). Bob 등록 201, 수정 200, 삭제 204	PASS	evidence/T17_1.png, evidence/T17_2.png

HTTP 응답 확인 기록

Network 나 API 클라이언트에서 직접 관찰한 값을 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회 ,로그인, 수정	200	200 / 해당 없음	evidence/T01.png
〃	200	200 / 해당 없음	evidence/T02_login200ok.png
〃	200	200 / 해당 없음	evidence/T06_put.png
새 글 등록	201 및 Location	201 / 해당 없음, Location: /api/posts/7	evidence/T05.png
본인 글 삭제	204 및 빈 본문	204 / 해당 없음, 빈 본문	evidence/T06_delete.png
입력 오류·추가 작성자 필드	400 BAD_INPUT	400 / BAD_INPUT	evidence/T13.png,
	400 BAD_INPUT	400 / BAD_INPUT	evidence/T12.png
잘못된 ID 형식	400 BAD_INPUT	400 / BAD_INPUT	evidence/T14_notfound.png
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	evidence/T03_wrongpw.png
〃	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	evidence/T03_wrongID.png
토큰 없음·헤더 형식 오류	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	evidence/T04_cmd_notoken.png
〃	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	evidence/T11.png
〃	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	evidence/T17_1.png
변조·클레임 오류	401 INVALID_TOKEN	401 / INVALID_TOKEN	evidence/T09.png
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	evidence/T10.png,

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	evidence/T10_2_browser_screen.png
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN	evidence/T07.png
	403 FORBIDDEN	403 / FORBIDDEN	evidence/T08.png
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	evidence/T14_notfound.png
토큰 없이 없는 글 수정	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED	evidence/overlap.png
Bob 이 Alice 글에 공백 제목으로 수정	**403 FORBIDDEN**	403 / FORBIDDEN	evidence/overlap.png

1. **보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거**

   토큰이 없거나 잘못된 보호 API 요청의 401 응답에는 모두 WWW-Authenticate: Bearer 헤더가 있었다. curl -i 결과(T04, T09, T10, T17)와 Network 의 Response headers(T11_2)에서 확인할수 있다.

   증거: evidence/T04_cmd_notoken.png, evidence/T09.png, evidence/T10.png, evidence/T17_1.png, evidence/T11_2.png

2. **401 과 403 에서 화면이 각각 어떻게 다른지 설명**

   401 이 오면 api.js 의 request()가 clearSession()을 호출해서 화면이 "로그아웃 상태"로 바뀌고, code 에 맞는 안내가 나온다. 만료는 "로그인 시간이 만료되었습니다. 다시 로그인하세요.", 토큰 없음은 "로그인이 필요합니다. 먼저 로그인하세요."가 표시됐다(T10_2_browser_screen.png, T11_2.png).
   403 은 누구인지는 확인된 상태라서 로그인을 풀지 않고, 로그인 상태를 유지한 채 본인 글만 수정, 삭제할 수 있다는 안내만 보여 준다. 화면에서는 canEdit()이 다른 사람 글의 수정, 삭제 버튼을 숨기기 때문에, 403 은 Bob 토큰으로 curl 요청을 직접 보내 확인했다

3. **실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과**

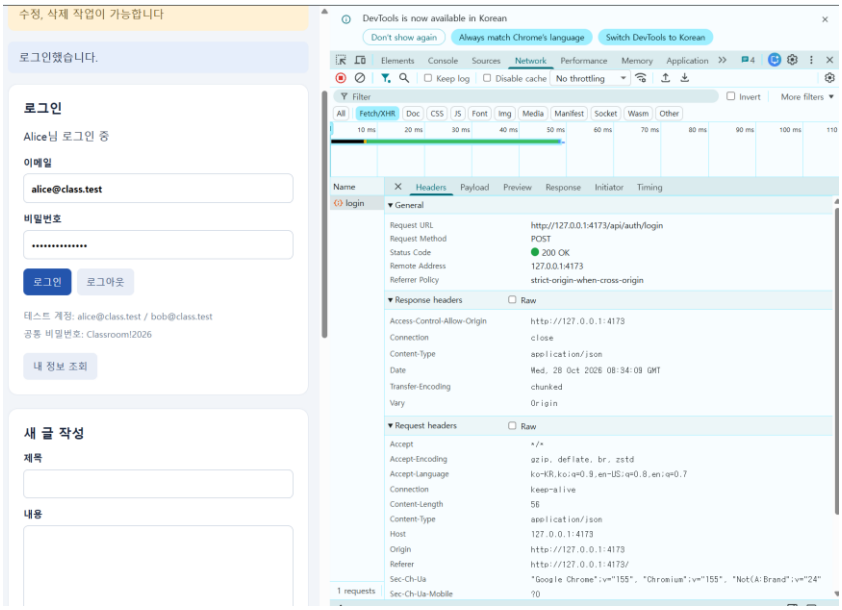
   T07(Bob 수정 403), T08(Bob 삭제 403), T09(변조 토큰 PUT, DELETE 401) 직후, 토큰 없이 GET /api/posts/7 을 보냈다. 세 번 모두 200 이었고 제목 "무 팔앙요", 내용 "무 팜미다", authorId 1, updatedAt 2026-10-28T08:43:18.866604Z 가 그대로였다. 실패한 요청은 글을 바꾸거나 지우지 못했다.

4. **토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과**

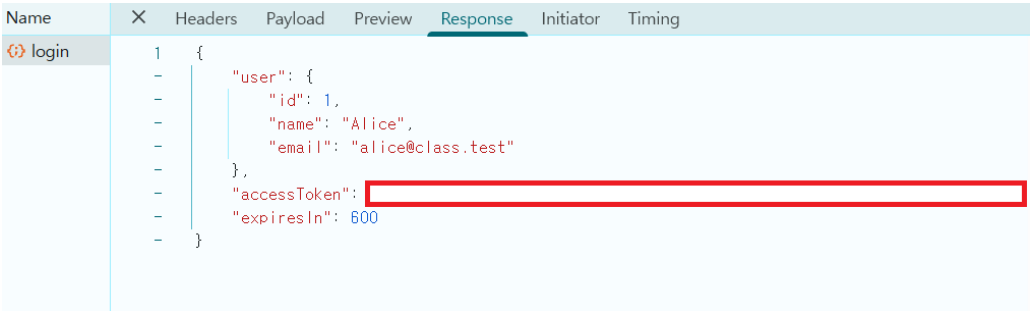
   정상 토큰(Alice)으로 PUT /api/posts/99999 를 보내면 404 /NOT_FOUND 였다. 토큰 없이 같은 요청을 보내면 서버가 인증을 먼저 하고 글이 있는지를 나중에 확인하기 때문에, 토큰이 없으면 글이 없어도 401 이 먼저 나온다.

테스트 증거

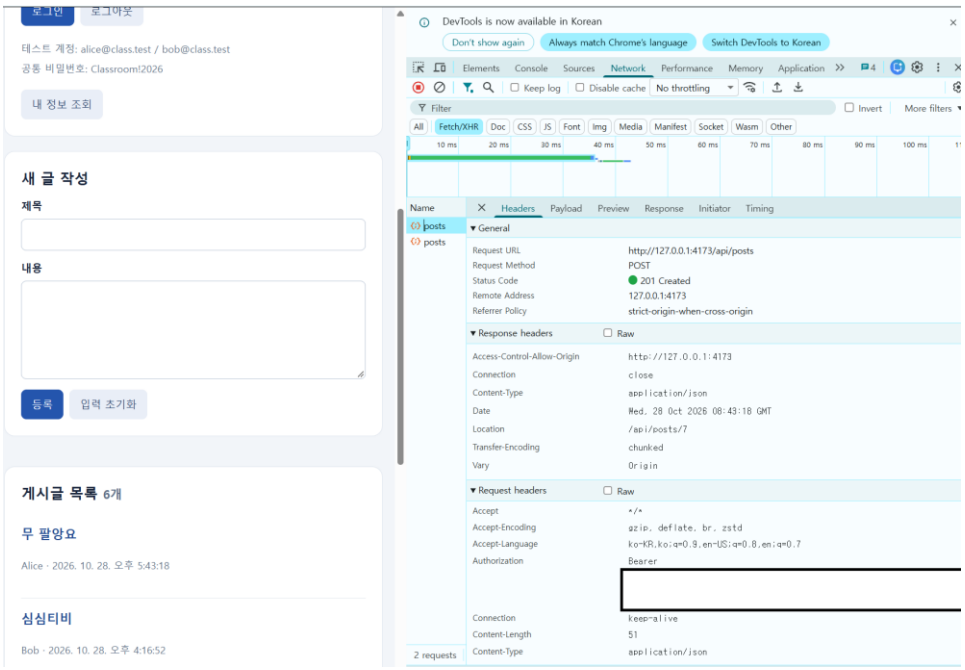
로그인과 토큰 전송



위 사진은 Alice 계정으로 로그인했을 때 POST /api/auth/login 이 200 OK 로 성공하고 화면에 "Alice 님 로그인 중"이 표시된 사진이다.

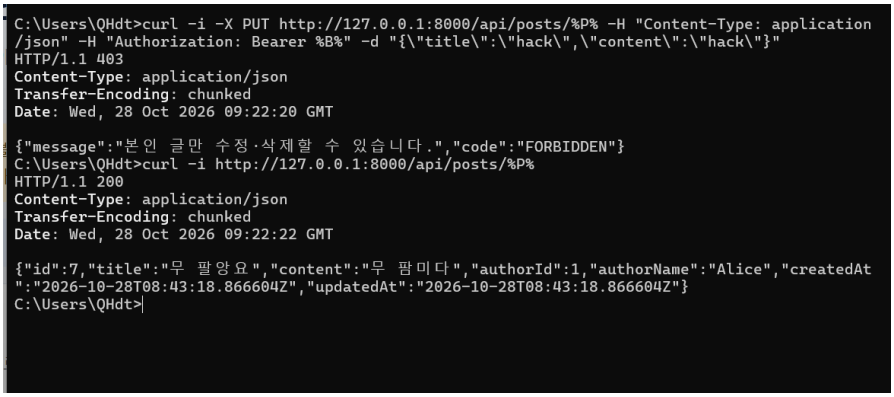


위 사진은 로그인 응답에 accessToken, expiresIn 600, user(id 1, Alice)가 담겨 온 것을 보여 주는 사진이다.

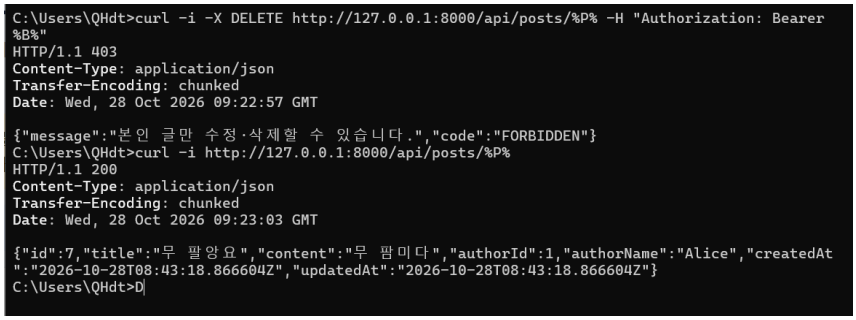


위 사진은 로그인한 Alice 가 글을 등록할 때 요청에 Authorization: Bearer 헤더가 붙고, 서버가 201 Created 와 Location /api/posts/7 로 응답한 사진이다.

다른 작성자의 수정과 삭제

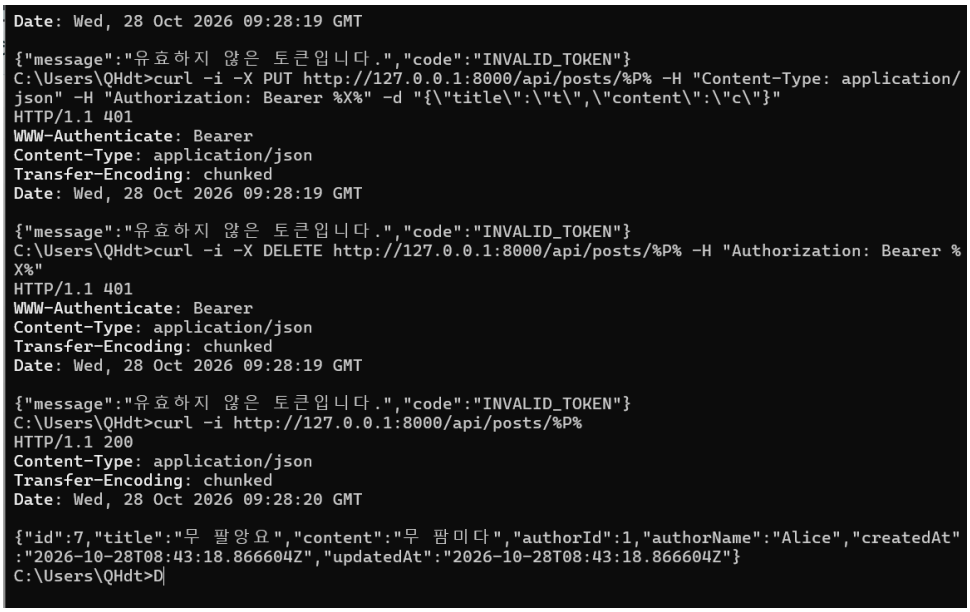


위 그림은 bob 토큰으로 Alice 글(7 번)을 수정하면 403 FORBIDDEN 으로 거부되고, 다시 조회해도 제목과 updatedAt 이 그대로인 것을 보여 주는 사진이다.

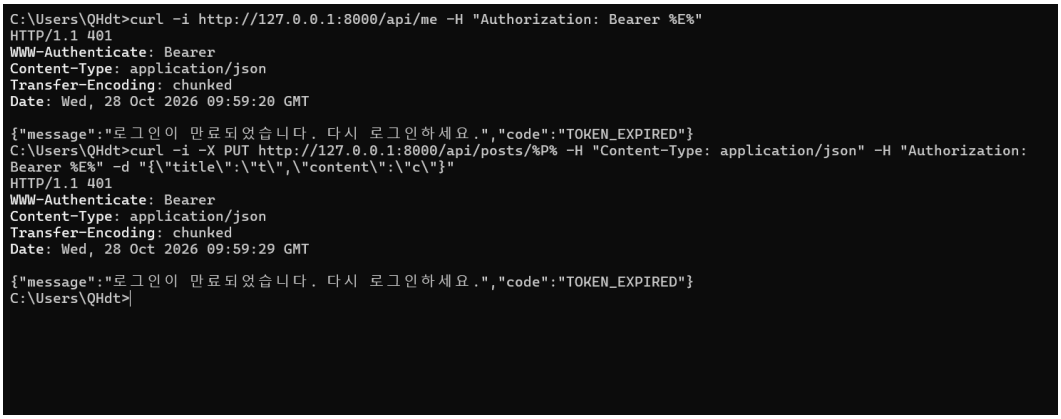


위 그림은 Bob 토큰으로 Alice 글(7 번)을 삭제하면 403 FORBIDDEN 으로 거부되고, 다시 조회해도 글이 남아 있는 것을 보여 주는 사진이다.

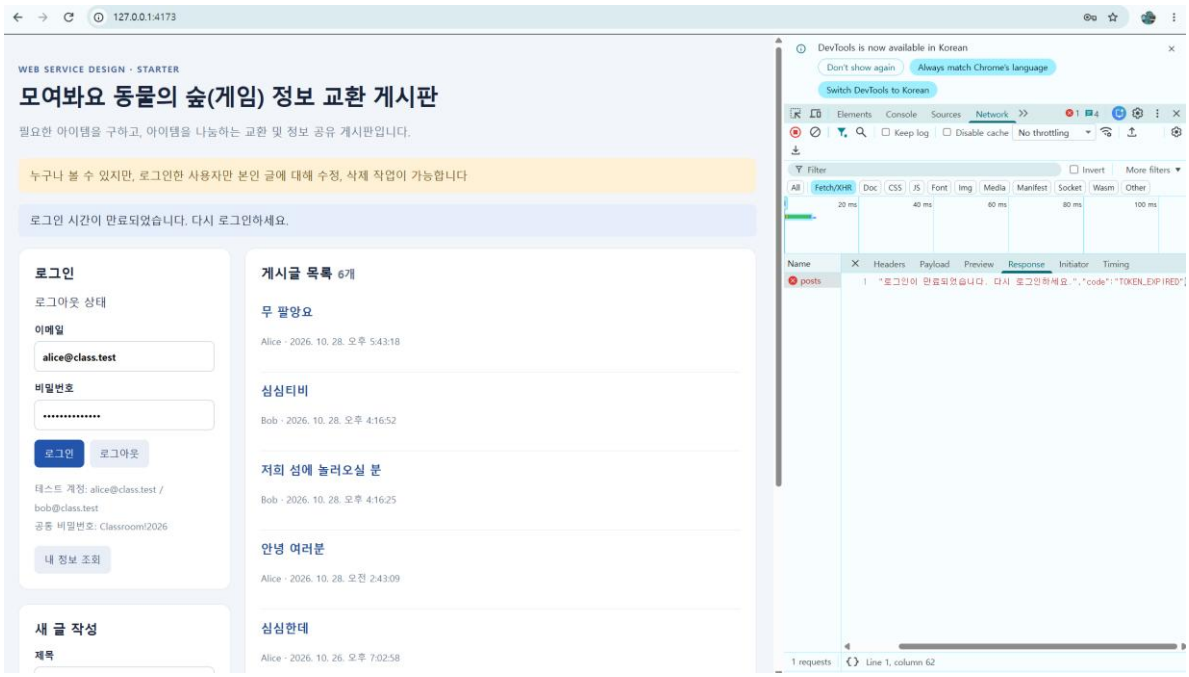
토큰 변조와 만료



위 그림은 서명 첫 글자를 바꾼 토큰으로 수정과 삭제를 보내면 401 INVALID_TOKEN 으로 거부되고, 글은 바뀌지 않은 것을 보여 주는 사진이다.



위 그림은 TTL 3 초로 받은 토큰이 만료된 뒤 /api/me 와 글 수정을 보내면 401 TOKEN_EXPIRED 로 거부되는 사진이다.



위 그림은 화면에서 만료된 토큰으로 요청했을 때 "로그인 시간이 만료되었습니다. 다시 로그인하세요." 안내가 뜨고 로그아웃 상태로 바뀐 사진이다.

자동 검사와 로컬 배포 증거

자동 검사: 미실행. 테스트는 위 T01~T17 수동 테스트로 확인했다.

빌드 성공 로그: evidence/Build_success.png (vite v8.3.4, built in 72ms)

4173 번 API 연결: evidence/T01.png

재시작 후 데이터 유지: evidence/T15_before_serverOff.png, evidence/T15_2_after_on.png

AI 활용과 참고 자료

AI 사용 여부: 사용

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
클로드-opus 5.5	학습, 설계, 생성, 디버깅, 테스트, 오타 수정 및 문장정리	backend-spring/.../AuthService.java, frontend-vue/src/api.js, frontend-vue/src/App.vue, T01~T17 테스트 절차, 보고서 1~6 장 전반	수정 후 반영 및 검증 후 그대로 반영

전체 사용 범위 요약:

이번 과제에서는 과제를 이해하기 위함부터 보고서 작성까지 전반적으로 활용하였다. 먼저 과제의 목적과 구성을 정리하고, 프론트엔드, 백엔드, DB 가 어떻게 연결되어 요청과 응답을 주고받는지, 그 사이에서 JWT 가 어디에 저장되고 어떤 요청에 함께 전송되는지를 공부하는 데 사용하였다. 이후 토큰 발급과 검증, 작성자 비교, 브라우저의 토큰 저장과 전송 코드의 초안을 받은 뒤, 각 코드가 어떤 역할을 하는지 함께 분석하면서 직접 수정하여 반영하였다. 빌드와 실행 중 발생한 오류의 원인을 찾는 과정과 T01 부터 T17 까지의 테스트 절차를 정리하는 데에도 활용하였고, 마지막으로 보고서의 오탈자 수정과 문장 정리, 보고서 형식의 어투로 변경하기에 사용하였다.

AI 가 제안한 코드와 문장은 그대로 사용하지 않고 수업 강의자료와 과제 안내 문서를 비교하며 확인한 뒤 필요한 부분을 수정하였으며, 모든 기능은 직접 실행하여 응답 코드를 확인하는 테스트로 검증하였다.

대표 사례 2 건

1 번)

도구 / 날짜: 클로드/ 261022

요청한 내용: 과제를 아직 파악하지 못해서 어떤 목적인지 그리고 어떤 구성이고 어떤걸 구현해야하는지 정리해줘 (프롬프트 요약)

AI 가 제안한 내용: 과제의 목적을 JWT 인증을 적용한 게시판 완성으로 정리하고, 서버에서 구현할 부분(토큰 발급, 토큰 검증, 작성자 권한 검사)과 화면에서 구현할 부분(로그인, 토큰 저장과 전송, 401 처리, 수정 버튼 표시)을 나누어 설명하였다. 또한 코드 구현, T01~T17 테스트와 캡처, 보고서 작성, GitHub 업로드와 LMS 제출로 이어지는 진행 순서를 함께 제안하였다. (출력 내용 요약)

실제 반영 위치: 과제 진행 순서 전반, AuthService.java 와 api.js 의 구현 범위 파악, 보고서 4 장 테스트 진행 계획

내가 확인하고 수정한 내용: 정리된 내용을 과제 공지, 구현 체크리스트, 테스트 시나리오 문서와 하나씩 비교하여 빠진 항목이 없는지 확인하였다. 또한 AI 가 안내한 작업 폴더 경로가 실제 컴퓨터에 없어서 실행이 되지 않았기 때문에, 직접 만든 폴더 경로로 바꾸어 진행하였다. 진행 순서는 시험 일정에 맞춰 여러 날로 나누어 조정하였다.

검증 방법과 결과: 구현 체크리스트의 항목과 실제 구현 내용을 대조하였고, 최종적으로 T01 부터 T17 까지의 테스트를 모두 수행하여 PASS 를 확인하였다.

2 번)

도구 / 날짜: 클로드/ 261027

요청한 내용: AuthService.java 에서 비어 있는 토큰 발급, 토큰 검증, 작성자 비교 함수와 api.js 의 로그인, 토큰 전송 부분의 코드 초안을 작성해 줘. 고쳐야 하는 부분만 주석으로 표시해 줘

AI 가 제안한 내용: 서버에서는 로그인에 성공한 사용자에게 사용자 id, 발급자, 사용 대상, 만료 시간을 담은 토큰을 서명하여 발급하고, 요청이 들어오면 서명과 만료 시간, 발급자와 사용 대상을 확인한 뒤 실패 이유에 따라 401 응답을 나누는 코드를 제안하였다. 또한 요청한 사용자와 글 작성자가 다르면 403 을 반환하는 코드도 함께 제안하였다. 화면에서는 로그인 응답의 토큰을 메모리에 저장하고, 요청마다 Bearer 헤더를 붙이며, 401 을 받으면 로그인 정보를 지우는 코드를 제안하였다. (코드 내용 요약)

실제 반영 위치: backend-spring/src/main/java/edu/classroom/board/AuthService.java / issueToken(), currentUserId(), assertOwner()

frontend-vue/src/api.js / login(), authorizationHeaders(), request()

내가 확인하고 수정한 내용: 처음에 제공된 시작 코드와 AI 초안을 합치는 과정에서 import 문과 변수가 중복으로 들어가고, 시작 코드에 있던 임시 반환 코드가 남아 빌드가 되지 않았다. 중복된 부분을 정리하고 임시 코드를 지운 뒤, 원래 있던 주석은 유지하면서 각 부분의 역할을 이해한 내용으로 주석을 수정하였다. 또한 7 주차 강의자료의 토큰 검증 코드와 비교하여 서명, 만료, 발급자, 사용 대상을 모두 확인하는지 점검하였다.

검증 방법과 결과: 수정 후 빌드가 정상적으로 완료되는 것을 확인하였다. 이후 T02 에서 로그인 시 200 과 토큰 발급, T07 과 T08 에서 Bob 의 수정과 삭제 요청이 403 으로 거부되는 것, T09 에서 변조한 토큰이 401 INVALID_TOKEN, T10 에서 만료된 토큰이 401 TOKEN_EXPIRED 로 거부되는 것을 확인하여 ai 의 출력물을 검증할 수 있었다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 3 주차 강의자료 :게시판 서비스 설계와 구현 27 쪽	보고서 4 장 HTTP 응답 확인 기록	200, 201, 204, 400, 404 상태 코드가 각각 어떤 상황에서 쓰이는지 참고하여 테스트 결과를 정리하였다.
웹서비스설계 7 주차 강의자료: JWT 접근 제어와 Swagger 64, 65, 68 쪽	AuthService.java / assertOwner(), BoardController.java / create(), update(), delete()	작성자는 서버가 인증된 사용자 id 로 정하고, 수정과 삭제 전에 글 작성자와 요청자를 비교해 다르면 403 FORBIDDEN 을 반환하는 구조를 참고하였다.
웹서비스설계 7 주차 강의자료 -JWT 접근 제어와 Swagger 53 쪽	AuthService.java / currentUserId()	강의자료의 토큰 검증 코드를 참고하여 서명, 만료, 발급자, 사용 대상을 확인하고 서명 방식이 HS256 인지 검사하도록 구현하였다.

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문은 사용자가 직접 수정해서 보낼 수 있는 값이다. 따라서 서버가 본문의 authorId 를 그대로 믿으면 Bob 이 authorId 값만 바꿔서 Alice 이름으로 글을 작성할 수 있게 되고, 작성자를 기준으로 하는 수정과 삭제 권한 검사도 의미가 없어진다.

이를 막기 위해 글을 등록할 때 작성자를 요청 본문에서 받지 않고, 서버가 토큰을 검증해서 확인한 사용자 id 로 저장하도록 구현하였다. 또한 title 과 content 외의 필드가 들어오면 400 BAD_INPUT 으로 요청을 거부하도록 하였다.

T12 에서 Bob 의 토큰으로 본문에 authorId 와 userId 를 각각 추가하여 요청한 결과, 두 경우 모두 400 BAD_INPUT 이 반환되었고 글은 생성되지 않았다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼을 숨기는 것은 화면에서만 적용되는 처리이다. 버튼이 보이지 않더라도 curl 이나 개발자 도구를 사용하면 수정, 삭제 요청을 서버에 직접 보낼 수 있기 때문에, 화면 처리만으로는 다른 사용자의 글이 변경되는 것을 막을 수 없다. 따라서 화면에서는 본인 글에만 버튼을 보여 주고, 실제 권한 검사는 서버에서 하도록 구현하였다.

서버에서는 AuthService 의 assertOwner() 함수가 권한을 검사한다. 수정과 삭제 요청이 들어오면 먼저 토큰으로 요청한 사용자를 확인하고, 글이 존재하는지 확인한 뒤, assertOwner()에서 요청한 사용자와 글 작성자가 같은지 비교한다. 두 값이 다르면 403 FORBIDDEN 을 반환하고 DB 는 변경하지 않는다.

T07 과 T08 에서 Bob 의 토큰으로 Alice 의 글에 수정, 삭제 요청을 직접 보낸 결과 모두 403 FORBIDDEN 이 반환되었고, 다시 조회했을 때 글 내용도 그대로 유지되어 있었다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰에 담긴 내용을 풀어서 읽기만 하는 것이다. JWT 의 내용 부분은 암호화되어 있지 않아 비밀 키 없이도 읽을 수 있기 때문에, decode 만으로는 토큰이 변조되었는지 또는 서버가 발급한 토큰이 맞는지 알 수 없다. 반면 verify 는 서버의 비밀 키로 서명을 확인하는 과정으로, 내용이 바뀌었거나 다른 키로 만들어진 토큰은 통과하지 못한다.

구현한 서버에서는 AuthService 의 currentUserId() 함수에서 verify 를 수행한다. Authorization 헤더에 토큰이 있는지 확인한 뒤 서버의 비밀 키로 서명을 검증하고, 서명 방식, 발급자와 사용 대상, 만료 시간을 확인한다.마지막으로 토큰에 들어 있는 사용자 id 가 실제로 존재하는 사용자인지까지 확인한다.

검증에 실패하면 토큰이 없는 경우는 401 TOKEN_REQUIRED, 만료된 경우는 401 TOKEN_EXPIRED, 그 외 잘못된 토큰은 401 INVALID_TOKEN 으로 구분하여 응답한다. T09 에서 서명을 변경한 토큰은 INVALID_TOKEN 으로, T10 에서 만료된 토큰은 TOKEN_EXPIRED 로 거부되는 것을 확인하였다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

토큰은 브라우저 저장소가 아닌 메모리에만 저장하도록 구현하였다. 따라서 세 경우 모두 로그인 상태가 해제되지만, 각각 바뀌는 부분은 다르다.

로그아웃은 현재 탭의 메모리에 저장된 토큰과 사용자 정보를 지우는 것이다. 이후 요청에는 토큰이 포함되지 않으며, T11 에서 로그아웃 후 내 정보를 조회했을 때 401 TOKEN_REQUIRED 가 반환되었다.

토큰 만료는 서버에서 판단한다. 만료 시간이 지난 토큰으로 요청하면 서버가 401 TOKEN_EXPIRED 를 반환하고, 화면에서는 이 응답을 받으면 로그인 정보를 지우고 다시 로그인하라는 안내를 표시한다. 새로고침은 페이지가 다시 실행되면서 메모리가 초기화되기 때문에 로그인이 해제된다. 다만 게시글은 서버의 DB 에 저장되어 있어 그대로 유지되며, 이는 T16 에서 확인하였다.

이미 복사된 토큰은 로그아웃이나 새로고침을 하더라도 만료 시간 전까지는 서버에서 계속 사용할 수 있다. 서버가 토큰을 따로 저장하지 않고 서명과 만료 시간만으로 판단하기 때문이다. 그래서 토큰의 유효 시간을 90 초로 짧게 설정하였다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

가장 어려웠던 오류는 서버 코드를 수정한 뒤 다시 빌드하는 과정에서 빌드가 실패한 것이다. 빌드를 실행하면 실행 파일(jar)을 새 파일로 교체할 수 없다는 오류가 발생하며 빌드가 중단되었다.

처음에는 방금 수정한 코드에 문법 오류가 있다고 생각하였다. 직전에 토큰 발급과 검증 함수를 작성하면서 같은 코드가 중복으로 들어간 실수가 있었기 때문이다. 그래서 중복된 부분을 정리하고 다시 빌드하였지만 같은 오류가 계속 발생하였다. 오류 내용을 다시 확인해 보니 코드를 컴파일하는 단계가 아니라 파일을 교체하는 단계에서 발생한 오류였다.

이후 AI 를 활용해 오류 내용을 확인하면서, Windows 에서는 실행 중인 프로그램이 사용하고 있는 파일을 덮어쓸 수 없다는 점을 알게 되었다. 이전에 다른 cmd 창에서 실행해 둔 서버가 같은 jar 파일을 계속 사용하고 있었기 때문에 빌드 과정에서 파일을 교체하지 못한 것이었다. 이를 통해 오류의 원인이 코드가 아니라 실행 환경에 있다는 것을 찾을 수 있었다.

원인을 확인한 뒤 실행 중이던 이전 서버를 종료하고 다시 빌드하였더니 정상적으로 완료되었다. 이후 새로 빌드한 서버를 실행하여 로그인 요청에 토큰이 정상적으로 발급되는 것을 확인하였고, T01 부터 T17 까지의 테스트를 진행하여 수정한 코드가 제대로 반영되었음을 확인하였다.

이 오류를 통해 빌드 오류가 항상 코드 때문에 발생하는 것은 아니라는 점을 알게 되었고, 이후에는 다시 빌드하기 전에 이전 서버가 실행 중인지 먼저 확인하였다.

현재 남은 문제와 다음에 개선할 점

로그아웃을 하더라도 이미 발급된 토큰을 서버에서 바로 막을 수 없다는 문제가 남아 있다. 토큰을 브라우저에서만 지우기 때문에, 토큰이 복사되어 있다면 만료 전까지는 사용할 수 있다. 다음에는 로그아웃한 토큰을 서버에서 기록하여 거부하는 방식이나, 유효 시간이 짧은 토큰과 재발급용 토큰을 함께 사용하는 방식을 적용해 보고자 한다.

또한 토큰을 메모리에만 저장하기 때문에 새로 고침을 하면 로그인이 해제되는 불편함이 있다. 과제 조건에 맞춘 방식이지만, 실제 서비스라면 편의성과 보안을 함께 고려하여 저장 방식을 설정해야한다고 생각한다.